

PUNTOS CLAVE

- Los dos aspectos fundamentales de la aritmética del computador son la forma de representar los números (el formato binario) y los algoritmos utilizados para realizar las operaciones aritméticas básicas (suma, resta, multiplicación y división). Estas dos consideraciones se aplican tanto a la aritmética de enteros como a la de coma flotante.
- Las cantidades en coma flotante se expresan como un número (mantisa) multiplicado por una constante (base) elevada a una potencia entera (exponente). Los números en coma flotante pueden utilizarse para representar cantidades muy grandes y muy pequeñas.
- La mayoría de los procesadores implementan la norma o estándar IEEE 754 para la representación de números y aritmética en coma flotante. Esta norma define el formato de 32 bits así como el de 64 bits.

Comenzamos nuestro estudio del procesador con la unidad aritmético-lógica (ALU). Tras una breve introducción a la ALU, el capítulo se centra en el aspecto más complejo de la misma: la aritmética del computador. Las funciones lógicas que forman parte de la ALU se describen en el Capítulo 10, y la implementación de funciones lógicas y aritméticas sencillas mediante lógica digital se describen en el Apéndice B del libro.

La aritmética de un computador se realiza normalmente con dos tipos de números muy diferentes: enteros y en coma flotante. En ambos casos, la representación elegida es un aspecto de diseño crucial que trataremos en primer lugar, seguido de una discusión sobre las operaciones aritméticas.

Este capítulo incluye diversos ejemplos que se resaltan en el texto mediante recuadros sombreados.

9.1. LA UNIDAD ARITMÉTICO-LÓGICA

La ALU es la parte del computador que realiza realmente las operaciones aritméticas y lógicas con los datos. El resto de los elementos del computador (unidad de control, registros, memoria, E/S) están principalmente para suministrar datos a la ALU, a fin de que esta los procese y para recuperar los resultados. Con la ALU llegamos al estudio de lo que puede considerarse el núcleo o esencia del computador.

Una unidad aritmético-lógica, y en realidad todos los componentes electrónicos del computador, se basan en el uso de dispositivos lógicos digitales sencillos que pueden almacenar dígitos binarios y realizar operaciones lógicas booleanas elementales. El Apéndice B explora, para el lector interesado, la implementación de circuitos lógicos digitales.

La Figura 9.1 indica, en términos generales, cómo se interconecta la ALU con el resto del procesador. Los datos se presentan a la ALU en registros, y en registros se almacenan los resultados de las operaciones producidos por la ALU. Estos registros son posiciones de memoria temporal internas al

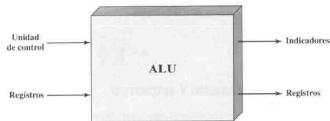


Figura 9.1. Entradas y salidas de la ALU.

procesador que están conectados a la ALU (véase por ejemplo la Figura 2.3). La ALU puede también activar indicadores (*flags*) como resultado de una operación.

Por ejemplo, un indicador de desbordamiento se pondrá a 1 si el resultado de una operación excede la longitud del registro en donde éste debe almacenarse. Los valores de los indicadores se almacenan también en otro registro dentro del procesador. La unidad de control proporciona las señales que gobiernan el funcionamiento de la ALU y la transferencia de datos dentro y fuera de la ALU.

9.2. REPRESENTACIÓN DE ENTEROS

En el sistema de numeración binaria¹, cualquier número puede representarse tan solo con los dígitos 1 y 0, el signo menos, y la **coma de la base** (que separa la parte entera de la decimal, el punto en los países anglosajones). Por ejemplo:

$$-1101,0101_2 = -13,3125_{10}$$

Sin embargo, para ser almacenados y procesados por un computador, no se tiene la posibilidad de disponer del signo y de la coma. Para representar los números solo pueden utilizarse dígitos 0 y 1. Si utilizáramos solo enteros no negativos, su representación sería inmediata.

Una palabra de ocho bits puede representar números desde 0 hasta 255, entre los que se encuentran:

00000000	=	0
00000001	=	1
00101001	=	41
10000000	=	128
11111111	=	255

¹ Para una revisión de los sistemas de numeración (decimal, binario, hexadecimal), consulte el Apéndice A.

En general, si una secuencia de n dígitos binarios $a_{n-1}a_{n-2}\dots a_1a_0$ es interpretada como un entero sin signo A , su valor es:

$$A = \sum_{i=0}^{n-1} 2^i a_i$$

REPRESENTACIÓN EN SIGNO Y MAGNITUD

Existen varias convenciones alternativas para representar números enteros tanto positivos como negativos. Todas ellas implican tratar el bit más significativo (el más a la izquierda) de la palabra como un bit de signo: si dicho bit es 0 el número es positivo, y si es 1, el número es negativo.

La forma más sencilla de representación que emplea un bit de signo es la denominada representación signo-magnitud. En una palabra de n bits, los $n-1$ bits de la derecha representan la magnitud del entero. Por ejemplo:

$$\begin{array}{l} +18 = 00010010 \\ -18 = 10010010 \end{array} \quad (\text{signo-magnitud})$$

El caso general puede expresarse como sigue:

$$\text{Signo y magnitud} \quad A = \begin{cases} \sum_{i=0}^{n-2} 2^i a_i & \text{si } a_{n-1} = 0 \\ -\sum_{i=0}^{n-2} 2^i a_i & \text{si } a_{n-1} = 1 \end{cases} \quad (9.1)$$

La representación signo-magnitud posee varias limitaciones. Una de ellas es que la suma y la resta requieren tener en cuenta tanto los signos de los números como sus magnitudes relativas para llevar a cabo la operación en cuestión. Esto debiera quedar claro con la discusión de la Sección 9.3. Otra limitación es que hay dos representaciones del número 0:

$$\begin{array}{l} +0_{10} = 00000000 \\ -0_{10} = 10000000 \end{array} \quad (\text{signo-magnitud})$$

Esto es un inconveniente porque es algo más difícil comprobar el valor 0 (una operación frecuentemente en los computadores) que si hubiera una sola representación.

Debido a estas limitaciones, raramente se usa la representación en signo-magnitud para implementar en la ALU las operaciones con enteros. En su lugar, el esquema más común es la representación en complemento a dos.

REPRESENTACIÓN EN COMPLEMENTO A DOS

Al igual que la de signo-magnitud, la representación en complemento a dos utiliza el bit más significativo como bit de signo, facilitando la comprobación de si el entero es positivo o negativo. Difiere

Tabla 9.1. Características de la representación numérica y la aritmética en complemento a dos.

Rango	-2^{n-1} hasta $2^{n-1}-1$
Número de representaciones del cero	Una
Negación	Realizar el complemento booleano de cada bit del correspondiente número positivo, y entonces sumar 1 al patrón de bits resultante visto como un entero sin signo.
Extensión de la longitud en bits	Añadir posiciones de bits a la izquierda rellenándolas con el valor del bit de signo original.
Regla de desbordamiento	Si se suman dos números con el mismo signo (ambos positivos o ambos negativos), solo se produce desbordamiento cuando el resultado tiene signo opuesto.
Regla de la sustracción	Para restar B de A , efectuar el complemento a dos de B y sumarlo a A .

de la representación signo-magnitud en la forma de interpretar los bits restantes. La Tabla 9.1 destaca las características clave de la representación y la aritmética en complemento a dos que serán elaboradas en esta sección y la siguiente.

La mayoría de los tratados sobre representación en complemento a dos se centran en las reglas para la obtención de los números negativos, sin pruebas formales de que el esquema utilizado «funcione». En su lugar, la presentación que hacemos de los números enteros en complemento a dos, en ésta y en la Sección 9.3, está basada en [DATT93], donde se sugiere que la representación en complemento a dos se entiende mejor definiéndola en términos de una suma ponderada de bits, como hicimos antes para las representaciones sin signo y en signo-magnitud. La ventaja de este tratamiento del tema está en que no queda duda alguna de si las reglas para las operaciones aritméticas con la notación en complemento a dos puedan no funcionar en algunos casos concretos.

Consideremos un entero de n bits, A , representado en complemento a dos. Si A es positivo, el bit de signo, a_{n-1} , es cero. Los restantes bits representan la magnitud del número de la misma forma que en la representación signo-magnitud; es decir:

$$A = \sum_{i=0}^{n-2} 2^i a_i \quad \text{para } A \geq 0$$

El número cero se identifica como positivo y tiene por tanto un bit de signo 0 y una magnitud de todo ceros. Podemos ver que el rango de los enteros positivos que pueden representarse es desde 0 (todos los bits de magnitud son 0) hasta $2^{n-1}-1$ (todos los bits de magnitud a 1). Cualquier número mayor requeriría más bits.

Ahora, para un número negativo A ($A < 0$), el bit de signo, a_{n-1} , es 1. Los $n-1$ bits restantes pueden tomar cualquiera de las 2^{n-1} combinaciones. Por lo tanto, el rango de los enteros negativos que pueden representarse es desde -1 hasta -2^{n-1} . Sería deseable asignar los bits de los enteros negativos de tal manera que su manipulación aritmética pueda efectuarse de una forma directa, similar a la de los enteros sin signo. En la representación sin signo, para calcular el valor de un entero a partir de su expresión en bits, el peso del bit más significativo es $+2^{n-1}$. Como veremos en la

Sección 9.3, para una representación con bit de signo resulta que las propiedades aritméticas deseadas se consiguen si el peso del bit más significativo es (-2^{n-1}) . Este es el convenio utilizado para la representación en complemento a dos, obteniéndose la siguiente expresión para los números negativos:

$$\text{Complemento a dos} \quad A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i \quad (9.2)$$

Para los enteros positivos $a_{n-1} = 0$, de forma que el término $-2^{n-1}a_{n-1} = 0$. Así pues, la ecuación (9.2) define la representación en complemento a dos, tanto para los números positivos como los negativos.

La Tabla 9.2 compara, para enteros de cuatro bits, las representaciones en signo-magnitud y en complemento a dos. Veremos que la representación en complemento a dos, aunque nos pueda resultar engorrosa, facilita las operaciones aritméticas más importantes, la suma y la resta. Por esta razón, es utilizada casi universalmente como representación de los enteros en los procesadores.

Tabla 9.2. Representaciones alternativas de los enteros de 4 bits.

Representación decimal	Representación signo-magnitud	Representación complemento a dos	Representación sesgada
+8	—	—	1111
+7	0111	0111	1110
+6	0110	0110	1101
+5	0101	0101	1100
+4	0100	0100	1011
+3	0011	0011	1010
+2	0010	0010	1001
+1	0001	0001	1000
+0	0000	0000	0111
-0	1000	—	—
-1	1001	1111	0110
-2	1010	1110	0101
-3	1011	1101	0100
-4	1100	1100	0011
-5	1101	1011	0010
-6	1110	1010	0001
-7	1111	1001	0000
-8	—	1000	—

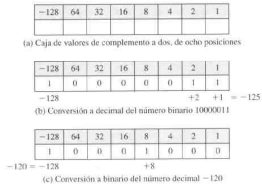


Figura 9.2. Utilización de la caja de valores para convertir entre binario en complemento a dos y decimal.

Una ilustración útil de la naturaleza de la representación en complemento a dos es una «caja» de valores, en la que el valor más a la derecha en la caja es 1 (2^0), y cada posición consecutiva hacia la izquierda tiene un valor doble al sumar (si el correspondiente bit es 1), hasta la posición más a la izquierda, cuyo valor es a restar. Como se puede ver en la Figura 9.2a, el número en complemento a dos más negativo representable es -2^{n-1} ; si cualquiera de los bits distintos del de signo es 1, este añade una cantidad positiva al número. También, está claro que un número negativo debe tener un 1 en la posición más a la izquierda, y un número positivo tendrá un cero en dicha posición. Por tanto, el número positivo mayor es un 0 seguido de todo unos, que es igual a $2^{n-1} - 1$.

El resto de la Figura 9.2 ilustra el uso de la caja de valores para convertir de complemento a dos a decimal, y de decimal a complemento a dos.

CONVERSIÓN ENTRE LONGITUDES DE BITS DIFERENTES

A veces se desea tomar un entero de n bits y almacenarlo en m bits, siendo $m > n$. Esto se resuelve fácilmente en la notación signo-magnitud: simplemente trasladando el bit de signo hasta la nueva posición más a la izquierda y rellenando con ceros.

+18	=	00010010	(signo-magnitud, 8 bits)
+18	=	0000000000010010	(signo-magnitud, 16 bits)
-18	=	11101110	(signo-magnitud, 8 bits)
-18	=	1000000000010010	(signo-magnitud, 16 bits)

Este procedimiento no funciona con los enteros negativos en complemento a dos. Utilizando el mismo ejemplo:

+18	=	00010010	(complemento a dos, 8 bits)
+18	=	000000000010010	(complemento a dos, 16 bits)
-18	=	11101110	(complemento a dos, 8 bits)
-32.658	=	100000001101110	(complemento a dos, 16 bits)

La penúltima línea anterior puede comprobarse fácilmente mediante la caja de valores de la Figura 9.2. La última línea puede verificarse utilizando la Ecuación (9.2) o mediante una caja de valores de 16 bits.

En su lugar, la regla para los enteros en complemento a dos es trasladar el bit de signo a la nueva posición más a la izquierda y completar con copias del bit de signo. Para números positivos, rellenar con ceros, y para negativos con unos. Así pues, se tiene:

-18	=	11101110	(complemento a dos, 8 bits)
-18	=	111111111101110	(complemento a dos, 16 bits)

Para ver que esta regla funciona, consideremos de nuevo una secuencia de n dígitos binarios $a_{n-1} a_{n-2} \dots a_1 a_0$ interpretada como entero A en complemento a dos, tal que su valor es:

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

Si A es positivo, la regla claramente funciona. Ahora, supongamos que A es negativo y que queremos construir una representación de m bits, con $m > n$. Entonces

$$A = -2^{m-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

Los dos valores deben ser iguales:

$$-2^{m-1} + \sum_{i=0}^{m-2} 2^i a_i = -2^{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

$$-2^{m-1} + \sum_{i=n-1}^{m-2} 2^i a_i = -2^{n-1}$$

$$2^{n-1} + \sum_{i=n-1}^{m-2} 2^i a_i = 2^{m-1}$$

$$1 + \sum_{i=0}^{n-2} 2^i + \sum_{i=n-1}^{m-2} 2^i a_i = 1 + \sum_{i=0}^{m-2} 2^i$$

$$\sum_{i=n-1}^{m-2} 2^i a_i = \sum_{i=n-1}^{m-2} 2^i$$

$$\Rightarrow a_{m-2} = \dots = a_{n-2} = a_{n-1} = 1$$

Al pasar de la primera a la segunda ecuación, se requiere que los $n - 1$ bits menos significativos no cambien entre las dos representaciones. Entonces, llegamos a la ecuación final, que solo es cierta si todos los bits desde la posición $n - 1$ hasta la $m - 2$ son 1. En consecuencia la regla funciona.

REPRESENTACIÓN EN COMA FIJA

Finalmente, mencionamos que la representación tratada en esta sección se denomina a veces de coma fija. Esto es porque la coma de la base (coma binaria) está fija y se supone que a la derecha del bit menos significativo. El programador puede utilizar la misma representación para fracciones binarias escalando los números de manera que la coma binaria esté implícitamente en alguna otra posición.

9.3. ARITMÉTICA CON ENTEROS

Esta sección examina funciones aritméticas comunes con números enteros representados en complemento a dos.

NEGACIÓN

En la representación signo-magnitud, la regla para obtener el opuesto de un entero es sencilla: invertir el bit de signo. En la notación de complemento a dos, la negación de un entero puede realizarse siguiendo las reglas:

1. Obtener el complemento booleano de cada bit del entero (incluyendo el bit de signo). Es decir, cambiar cada 1 por 0, y cada 0 por 1.
2. Tratando el resultado como un entero binario sin signo, sumarle 1.

Este proceso en dos etapas se denomina **transformación a complemento a dos**, u obtención del complemento a dos de un entero. Por ejemplo:

+18	=	00010010	(complemento a dos)
complemento bit a bit	=	11101101	
	+	1	
		11101110	= -18

Como es de esperar, el opuesto del opuesto es el propio número:

-18	=	11101110	(complemento a dos)
complemento bit a bit	=	00010001	
		+ 1	
		00010010	= +18

Podemos demostrar la validez de la operación que acabamos de describir utilizando la definición de representación en complemento a dos dada en la Ecuación (9.2). De nuevo interpretamos una secuencia de n dígitos binarios $a_{n-1} a_{n-2} \dots a_1 a_0$ como un entero A en complemento a dos, tal que su valor es:

$$A = -2^{n-1}a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

Ahora se construye el complemento bit a bit, $\overline{a_{n-1} a_{n-2} \dots a_0}$, y tratándolo como un entero sin signo, se le suma 1. Finalmente, se interpreta la secuencia resultante de n bits como un entero B en complemento a dos, de manera que su valor es

$$B = -2^{n-1} \overline{a_{n-1}} + 1 + \sum_{i=0}^{n-2} 2^i \overline{a_i}$$

Ahora queremos que $A = -B$, lo que significa que $A + B = 0$. Esto se comprueba fácilmente:

$$\begin{aligned}
 A + B &= -(a_{n-1} + \overline{a_{n-1}}) 2^{n-1} + 1 + \left(\sum_{i=0}^{n-2} 2^i (a_i + \overline{a_i}) \right) \\
 &= -2^{n-1} + 1 + \left(\sum_{i=0}^{n-2} 2^i \right) \\
 &= -2^{n-1} + 1 + (2^{n-1} - 1) \\
 &= -2^{n-1} + 2^{n-1} = 0
 \end{aligned}$$

El desarrollo anterior supone que podemos primero tratar el complemento de A bit a bit como entero sin signo al objeto de sumarle 1, y entonces tratar el resultado como un entero en complemento a dos. Hay dos casos especiales a tener en cuenta. En primer lugar, consideremos que $A = 0$. En este caso, para una representación con ocho bits:

0	=	00000000	(complemento a dos)
complemento bit a bit	=	11111111	
		+ 1	
		10000000	= 0

Hay un *acarreo* de la posición de bit más significativa, que es ignorado. El resultado es que la negación u opuesto del 0 es 0, como debe ser.

El segundo caso especial es más problemático. Si generamos el opuesto de la combinación de bits consistente en un 1 seguido de $n - 1$ ceros, se obtiene de nuevo el mismo número. Por ejemplo, para palabras de ocho bits,

$$\begin{array}{rcl}
 -128 & = & 10000000 \quad (\text{complemento a dos}) \\
 \text{complemento bit a bit} & = & 01111111 \\
 & + & 1 \\
 & = & 10000000 = -128
 \end{array}$$

Esta anomalía debe evitarse. El número de combinaciones diferentes en una palabra de ocho bits es 2^8 , un número par. Con ellas queremos representar enteros positivos, negativos y el 0. Cuando se representa el mismo número de enteros positivos que de negativos (en signo-magnitud) resultan dos representaciones distintas del 0. Si hay solo una representación del 0 (en complemento a dos), entonces debe haber un número desigual de números positivos que de negativos representados. En el caso del complemento a dos, hay una representación de n bits para el -2^{n-1} , pero no para el $+2^{n-1}$.

SUMA Y RESTA

La suma en complemento a dos se ilustra en la Figura 9.3. La suma se efectúa igual que si los números fuesen enteros sin signo. Los cuatro primeros ejemplos muestran operaciones correctas. Si el resultado de la operación es positivo, se obtiene un número positivo en forma de complemento a dos, que tiene la misma forma como entero sin signo. Si el resultado de la operación es negativo, conseguimos un número negativo en forma de complemento a dos. Obsérvese que, en algunos casos, hay un bit acarreo más allá del final de la palabra (sombreado en la figura). Este bit se ignora.

En cualquier suma, el resultado puede que sea mayor que el permitido por la longitud de palabra que está utilizando. Esta condición se denomina **desbordamiento** (*overflow*). Cuando ocurre un

$ \begin{array}{r} 1001 = -7 \\ +0101 = 5 \\ \hline 1110 = -2 \\ \text{(a) } (-7) + (+5) \end{array} $	$ \begin{array}{r} 1100 = -4 \\ +0100 = 4 \\ \hline 0000 = 0 \\ \text{(b) } (-4) + (+4) \end{array} $
$ \begin{array}{r} 0011 = 3 \\ +0100 = 4 \\ \hline 0111 = 7 \\ \text{(c) } (+3) + (+4) \end{array} $	$ \begin{array}{r} 1100 = -4 \\ +1111 = -1 \\ \hline 1011 = -5 \\ \text{(d) } (-4) + (-1) \end{array} $
$ \begin{array}{r} 0101 = 5 \\ +0100 = 4 \\ \hline 1001 = \text{Desbordamiento} \\ \text{(e) } (+5) + (+4) \end{array} $	$ \begin{array}{r} 1001 = -7 \\ +1010 = -6 \\ \hline 0011 = \text{Desbordamiento} \\ \text{(f) } (-7) + (-6) \end{array} $

Figura 9.3. Suma de números representados en complemento a dos.

desbordamiento, la ALU debe indicarlo para que no se intente utilizar el resultado obtenido. Para detectar el desbordamiento se debe observar la siguiente regla:

REGLA DE DESBORDAMIENTO: al sumar dos números, y ambos son o bien positivos o negativos, se produce desbordamiento si y solo si el resultado tiene signo opuesto.

Las Figuras 9.3e y f muestran ejemplos de desbordamiento. Obsérvese que el desbordamiento puede ocurrir habiéndose producido o no acarreo.

La resta se trata también fácilmente con la siguiente regla:

REGLA DE LA RESTA: para sustraer un número (el sustraendo) de otro (minuendo), se obtiene el complemento a dos del sustraendo y se le suma al minuendo.

Así pues, la resta se consigue usando la suma, como se muestra en la Figura 9.4. Los dos últimos ejemplos demuestran que también es aplicable la regla de desbordamiento anterior.

Una ilustración gráfica como la mostrada en la Figura 9.5 [BENH92] proporciona una visión más palpable de la suma y la resta en complemento a dos. Los círculos (mitades superiores de la figura) se obtienen a partir de los correspondientes segmentos lineales de números (mitades inferiores), juntando los extremos. Observe que cuando los números se trazan en el círculo, el complemento a dos de cualquier número es el horizontalmente opuesto del mismo (indicado mediante líneas horizontales discontinuas). Comenzando en cualquier número del círculo, al sumarle un positivo k (o restarle un

$\begin{array}{r} 0010 = 2 \\ +1001 = -7 \\ \hline 1011 = -5 \end{array}$ (a) $M = 2 = 0010$ $S = 7 = 0111$ $-S = 1001$	$\begin{array}{r} 0101 = 5 \\ +1110 = -2 \\ \hline 0011 = 3 \end{array}$ (b) $M = 5 = 0101$ $S = 2 = 0010$ $-S = 1110$
$\begin{array}{r} 1011 = -5 \\ +1110 = -2 \\ \hline 1001 = -7 \end{array}$ (c) $M = -5 = 1011$ $S = 2 = 0010$ $-S = 1110$	$\begin{array}{r} 0101 = 5 \\ +0010 = 2 \\ \hline 0111 = 7 \end{array}$ (d) $M = 5 = 0101$ $S = -2 = 1110$ $-S = 0010$
$\begin{array}{r} 0111 = 7 \\ +0111 = 7 \\ \hline 1110 = \text{Desbordamiento} \end{array}$ (e) $M = 7 = 0111$ $S = -7 = 1001$ $-S = 0111$	$\begin{array}{r} 1010 = -6 \\ +1100 = -4 \\ \hline 0110 = \text{Desbordamiento} \end{array}$ (f) $M = -6 = 1010$ $S = 4 = 0100$ $-S = 1100$

Figura 9.4. Substracción de números en la notación de complemento a dos ($M - S$).

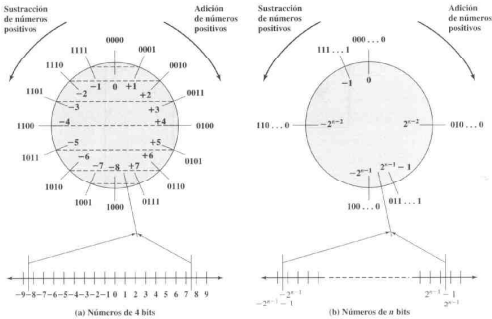


Figura 9.5. Descripción geométrica de los enteros en complemento a dos.

negativo k) nos desplazamos k posiciones en el sentido de las agujas del reloj. Restarle un positivo k (o sumarle un negativo k) equivale a desplazarse k posiciones en sentido contrario a las agujas del reloj. Si la operación realizada hace que se sobrepase el punto en que se juntaron los extremos del segmento, el resultado es incorrecto (desbordamiento).

Todos los ejemplos de las Figuras 9.3 y 9.4 pueden trazarse fácilmente en el círculo de la Figura 9.5.

La Figura 9.6 sugiere los caminos de datos y elementos hardware necesarios para realizar sumas y restas. El elemento central es un sumador binario, al que se presentan los números a sumar y produce una suma y un indicador de desbordamiento. El sumador binario trata los dos números como enteros sin signo (una implementación lógica de un sumador se da en el Apéndice B de este libro). Para sumar, los números se presentan al sumador desde dos registros, designados en este caso registros A y B. El resultado es normalmente almacenado en uno de estos registros en lugar de un tercero. La indicación de desbordamiento se almacena en un indicador o biestable de desbordamiento (OF: *Overflow Flag*) de 1 bit (0 = no desbordamiento; 1 = desbordamiento). Para la resta, el substraendo (registro B) se pasa a través de un complementador, de manera que el valor que se presenta al sumador sea el complemento a dos de B.